

Lie Groups and Lie Algebras for Robotics

Rotations and Rigid-Body Transformations

Jason Wakim Richards

September 20, 2026

Contents

1	Introduction	1
1.1	Notation and conventions	1
2	Coordinate Frames	1
3	SO(3): Rotations	2
3.1	Why is SO(3) a Lie group?	2
3.2	Axis-angle representation	3
3.3	The Lie algebra and hat operator	3
3.4	The exponential map	3
3.5	The logarithmic map	3
4	SE(3): Rigid-Body Transformations	4
4.1	The SE(3) group	4
4.2	Homogeneous Coordinates	4
4.3	SE(3) Inverse	5
4.4	The Lie algebra $\mathfrak{se}(3)$	5
4.5	SE(3) Exponential Map	5
4.6	SE(3) Logarithmic Map	5
5	The Big Picture	6
5.1	Why is this useful in robotics?	6
6	Active and Passive Transformations	7
6.1	Left and right perturbations	7
7	Implementation with Eigen	8
8	Exercises	9
9	C++ Exercises	11
10	Further Reading	13

1 Introduction

This document introduces the Lie groups used to represent three-dimensional rotations and rigid-body transformations in robotics. It develops the geometry of $SO(3)$ and $SE(3)$, their Lie algebras, and the exponential and logarithmic maps that connect them. Worked examples and implementation notes show how these ideas are used in estimation, control, computer vision, and simultaneous localisation and mapping (SLAM).

The reader should be comfortable with vectors, matrices, matrix multiplication, determinants, and basic calculus. By the end of the document, the reader should be able to:

- interpret frame-labelled rotations and transformations;
- distinguish a Lie group element from its Lie algebra coordinates;
- compute and interpret the exponential and logarithmic maps of $SO(3)$ and $SE(3)$;
- compose and invert rigid-body transformations; and
- distinguish active and passive transformations from left and right perturbations.

1.1 Notation and conventions

Vectors are bold lowercase symbols, matrices are bold uppercase symbols, and all coordinate frames are right-handed. The notation ${}^B\mathbf{R}_A$ denotes the rotation that maps vector coordinates from frame A to frame B . Similarly, ${}^B\mathbf{T}_A$ maps homogeneous point coordinates from frame A to frame B . Angles are measured in radians.

The groups are written $SO(3)$ and $SE(3)$, while their Lie algebras are written $\mathfrak{so}(3)$ and $\mathfrak{se}(3)$. The hat operator $(\cdot)^\wedge$ maps vector coordinates to a Lie algebra matrix; the vee operator $(\cdot)^\vee$ is its inverse. This document uses Exp and Log for vector-valued Lie-group maps and exp and log for matrix functions.

2 Coordinate Frames

Consider two coordinate frames, A and B . Let ${}^B\mathbf{R}_A \in SO(3)$ map vector coordinates from frame A to frame B , and let ${}^B\mathbf{t}_A \in \mathbb{R}^3$ be the position of the origin of frame A , measured from the origin of frame B and expressed in frame B . A point with coordinates ${}^A\mathbf{p}$ in frame A has coordinates ${}^B\mathbf{p}$ in frame B given by

$${}^B\mathbf{p} = {}^B\mathbf{R}_A {}^A\mathbf{p} + {}^B\mathbf{t}_A \quad (2.1)$$

This coordinate transformation contains both a rotation and a translation. The quantities in (2.1) are:

- ${}^B\mathbf{p}$ point expressed in frame B ;
- ${}^B\mathbf{R}_A$ coordinate rotation from frame A to frame B ;
- ${}^A\mathbf{p}$ point expressed in frame A ;
- ${}^B\mathbf{t}_A$ position of the A -origin, expressed in frame B .

It is important to get a clear understanding of what the variables mean and what frames they are in. Figure 1 shows a diagram of the different vectors.

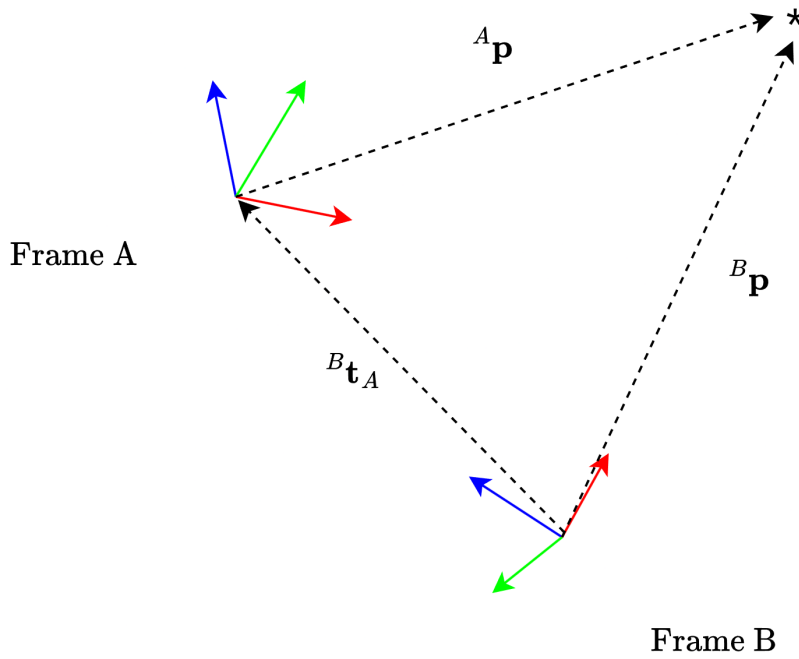


Figure 1: The same geometric point expressed in frames A and B . The translation ${}^B\mathbf{t}_A$ points from the B -origin to the A -origin and is expressed in frame B .

3 SO(3): Rotations

A point is represented by a vector in $\mathbb{R}^3$; a rotation of that point is represented by a matrix

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}. \quad (3.1)$$

Definition 3.1. The special orthogonal group in three dimensions is

$$\text{SO}(3) = \{ \mathbf{R} \in \mathbb{R}^{3 \times 3} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I}_3, \det(\mathbf{R}) = 1 \}. \quad (3.2)$$

Its elements are the proper rotation matrices.

Orthogonality implies $\mathbf{R}^{-1} = \mathbf{R}^\top$. The determinant constraint excludes reflections. Although a general 3×3 matrix has nine entries, these constraints leave only three degrees of freedom.

3.1 Why is SO(3) a Lie group?

The group operation is matrix multiplication. For $\mathbf{R}_1, \mathbf{R}_2 \in \text{SO}(3)$, closure follows because

$$(\mathbf{R}_2 \mathbf{R}_1)^\top (\mathbf{R}_2 \mathbf{R}_1) = \mathbf{I}_3, \quad (3.3)$$

$$\det(\mathbf{R}_2 \mathbf{R}_1) = \det(\mathbf{R}_2) \det(\mathbf{R}_1) = 1. \quad (3.4)$$

The identity is $\mathbf{I}_3$, every element has inverse $\mathbf{R}^\top$, and associativity is inherited from matrix multiplication. Therefore $\text{SO}(3)$ is a group. It is also a smooth manifold, and multiplication and inversion are smooth operations. A group with this additional smooth structure is a *Lie group*.

3.2 Axis--angle representation

Every three-dimensional rotation can be represented by a unit axis $\mathbf{u}$ and an angle θ in radians:

$$\|\mathbf{u}\| = 1, \quad \boldsymbol{\phi} = \theta \mathbf{u}. \quad (3.5)$$

The vector $\boldsymbol{\phi}$ is called a *rotation vector*. Its magnitude is θ and, when $\theta \neq 0$, its direction is the rotation axis. This representation is not globally unique: angles differing by 2π can describe the same rotation, the axis is undefined at $\theta = 0$, and the representation is ambiguous at $\theta = \pi$. A principal logarithm convention normally chooses $\theta \in [0, \pi]$.

3.3 The Lie algebra and hat operator

The Lie algebra $\mathfrak{so}(3) = T_1\text{SO}(3)$ is the tangent space of $\text{SO}(3)$ at the identity. Its elements are skew-symmetric matrices. The hat operator maps vector coordinates into this matrix space:

$$(\cdot)^\wedge : \mathbb{R}^3 \rightarrow \mathfrak{so}(3), \quad \boldsymbol{\phi}^\wedge = \begin{bmatrix} 0 & -\phi_z & \phi_y \\ \phi_z & 0 & -\phi_x \\ -\phi_y & \phi_x & 0 \end{bmatrix}. \quad (3.6)$$

It satisfies

$$\boldsymbol{\phi}^\wedge \mathbf{x} = \boldsymbol{\phi} \times \mathbf{x}. \quad (3.7)$$

The vee operator reverses this mapping: $(\boldsymbol{\phi}^\wedge)^\vee = \boldsymbol{\phi}$. The Lie bracket is the matrix commutator $[\mathbf{A}, \mathbf{B}] = \mathbf{AB} - \mathbf{BA}$, and it corresponds to the vector cross product:

$$[\boldsymbol{\phi}^\wedge, \boldsymbol{\psi}^\wedge] = (\boldsymbol{\phi} \times \boldsymbol{\psi})^\wedge. \quad (3.8)$$

3.4 The exponential map

The exponential map converts rotation-vector coordinates into a rotation:

$$\text{Exp} : \mathbb{R}^3 \rightarrow \text{SO}(3), \quad \text{Exp}(\boldsymbol{\phi}) = \exp(\boldsymbol{\phi}^\wedge). \quad (3.9)$$

The matrix exponential is defined by its power series. Since $(\boldsymbol{\phi}^\wedge)^3 = -\theta^2 \boldsymbol{\phi}^\wedge$ for $\theta = \|\boldsymbol{\phi}\|$, the series reduces to Rodrigues' formula:

$$\text{Exp}(\boldsymbol{\phi}) = \mathbf{I}_3 + \frac{\sin \theta}{\theta} \boldsymbol{\phi}^\wedge + \frac{1 - \cos \theta}{\theta^2} (\boldsymbol{\phi}^\wedge)^2. \quad (3.10)$$

At $\theta = 0$, the scalar coefficients are interpreted by continuity. For small angles,

$$\text{Exp}(\boldsymbol{\phi}) \approx \mathbf{I}_3 + \boldsymbol{\phi}^\wedge. \quad (3.11)$$

In numerical software, use series expansions near zero rather than directly evaluating ratios that have small denominators.

3.5 The logarithmic map

The principal logarithmic map returns rotation-vector coordinates:

$$\text{Log} : \text{SO}(3) \rightarrow \mathbb{R}^3, \quad \text{Log}(\mathbf{R}) = (\log \mathbf{R})^\vee = \phi. \quad (3.12)$$

For $0 < \theta < \pi$,

$$\theta = \cos^{-1} \left(\frac{\text{tr}(\mathbf{R}) - 1}{2} \right), \quad (3.13)$$

$$\phi^\wedge = \frac{\theta}{2 \sin \theta} (\mathbf{R} - \mathbf{R}^\top). \quad (3.14)$$

Floating-point implementations should clamp the argument of $\cos^{-1}$ to $[-1, 1]$. The cases near $\theta = 0$ and $\theta = \pi$ require dedicated branches because (3.14) becomes numerically ill-conditioned.

Example 3.1. For a rotation of $\pi/2$ about the z -axis,

$$\phi = \begin{bmatrix} 0 & 0 & \pi/2 \end{bmatrix}^\top, \quad \text{Exp}(\phi) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (3.15)$$

Applying this matrix to $[1 \ 0 \ 0]^\top$ produces $[0 \ 1 \ 0]^\top$.

4 SE(3): Rigid-Body Transformations

4.1 The SE(3) group

Definition 4.1. The special Euclidean group in three dimensions is

$$\text{SE}(3) = \left\{ \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \mid \mathbf{R} \in \text{SO}(3), \mathbf{t} \in \mathbb{R}^3 \right\}. \quad (4.1)$$

Each element represents a three-dimensional rigid-body transformation.

The rotation block has three degrees of freedom and the translation block has three more, so $\text{SE}(3)$ is a six-dimensional Lie group.

4.2 Homogeneous Coordinates

Introduce the homogeneous coordinates

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix} \quad (4.2)$$

so that rotation and translation can be applied by one matrix multiplication:

$${}^B\tilde{\mathbf{p}} = {}^B\mathbf{T}_A {}^A\tilde{\mathbf{p}}. \quad (4.3)$$

Transformations compose by matching adjacent frame labels. For example,

$${}^C\mathbf{T}_A = {}^C\mathbf{T}_B {}^B\mathbf{T}_A. \quad (4.4)$$

The rightmost transformation acts first. This frame-cancellation rule is a useful check on multiplication order.

4.3 SE(3) Inverse

$${}^B\mathbf{T}_A = \begin{bmatrix} {}^B\mathbf{R}_A & {}^B\mathbf{t}_A \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (4.5)$$

has inverse

$${}^A\mathbf{T}_B = ({}^B\mathbf{T}_A)^{-1} = \begin{bmatrix} ({}^B\mathbf{R}_A)^T & -({}^B\mathbf{R}_A)^T {}^B\mathbf{t}_A \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (4.6)$$

The inverse translation is not simply $-{}^B\mathbf{t}_A$ because it must be expressed in frame A .

4.4 The Lie algebra $\mathfrak{se}(3)$

The Lie algebra $\mathfrak{se}(3) = T_1 \text{SE}(3)$ is represented using the six-vector

$$\xi = \begin{bmatrix} \rho \\ \phi \end{bmatrix} \in \mathbb{R}^6, \quad \xi^\wedge = \begin{bmatrix} \phi^\wedge & \rho \\ \mathbf{0}_{1 \times 3} & 0 \end{bmatrix} \in \mathfrak{se}(3). \quad (4.7)$$

This document orders translation before rotation in ξ . Other texts use the reverse ordering, so implementations must state their convention. The quantity ρ is a translational *tangent coordinate*; it is not generally equal to the translation $\mathbf{t}$ in a transformation matrix. For a velocity twist, the analogous components are commonly written $[\mathbf{v}^T \ \omega^T]^T$ and have units of metres per second and radians per second.

4.5 SE(3) Exponential Map

The vector-valued exponential map is

$$\text{Exp} : \mathbb{R}^6 \rightarrow \text{SE}(3), \quad \text{Exp}(\xi) = \exp(\xi^\wedge) = \begin{bmatrix} \text{Exp}(\phi) & \mathbf{J}(\phi)\rho \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}. \quad (4.8)$$

Here $\theta = \|\phi\|$ and the $\text{SO}(3)$ left Jacobian is

$$\mathbf{J}(\phi) = \mathbf{I}_3 + \frac{1 - \cos \theta}{\theta^2} \phi^\wedge + \frac{\theta - \sin \theta}{\theta^3} (\phi^\wedge)^2. \quad (4.9)$$

Therefore $\mathbf{t} = \mathbf{J}(\phi)\rho$: rotation and translation are coupled by the exponential map. Near zero, $\mathbf{J}(\phi) \approx \mathbf{I}_3 + \frac{1}{2}\phi^\wedge + \frac{1}{6}(\phi^\wedge)^2$.

4.6 SE(3) Logarithmic Map

For $\mathbf{T} = [\mathbf{R} \ \mathbf{t}; \ \mathbf{0}^T \ 1]$, first compute $\phi = \text{Log}(\mathbf{R})$ and then

$$\rho = \mathbf{J}(\phi)^{-1}\mathbf{t}. \quad (4.10)$$

The resulting vector map and its relationship to the matrix logarithm are

$$\text{Log}(\mathbf{T}) = \begin{bmatrix} \boldsymbol{\rho} \\ \phi \end{bmatrix} = \boldsymbol{\xi}, \quad \log(\mathbf{T}) = \boldsymbol{\xi}^\wedge. \quad (4.11)$$

For nonzero θ away from singular cases,

$$\mathbf{J}^{-1} = \mathbf{I}_3 - \frac{1}{2}\phi^\wedge + \left(\frac{1}{\theta^2} - \frac{1 + \cos \theta}{2\theta \sin \theta} \right) (\phi^\wedge)^2. \quad (4.12)$$

Near zero, use $\mathbf{J}^{-1} \approx \mathbf{I}_3 - \frac{1}{2}\phi^\wedge + \frac{1}{12}(\phi^\wedge)^2$. As with the $\text{SO}(3)$ logarithm, robust software needs dedicated handling near $\theta = 0$ and $\theta = \pi$.

Example 4.1. Let ${}^B\mathbf{R}_A$ be a 90° rotation about z and let ${}^B\mathbf{t}_A = [1 \ 2 \ 0]^\top$. A point ${}^A\mathbf{p} = [1 \ 0 \ 0]^\top$ becomes

$${}^B\mathbf{p} = {}^B\mathbf{R}_A {}^A\mathbf{p} + {}^B\mathbf{t}_A = \begin{bmatrix} 1 \\ 3 \\ 0 \end{bmatrix}. \quad (4.13)$$

The frame labels make clear that the output coordinates are expressed in B .

5 The Big Picture

The relationships developed above can be summarised as

$$\mathbb{R}^3 \xrightleftharpoons[\begin{smallmatrix} (\cdot)^\vee \end{smallmatrix}]{\begin{smallmatrix} (\cdot)^\wedge \end{smallmatrix}} \mathfrak{so}(3) \xrightleftharpoons[\log]{\exp} \text{SO}(3), \quad (5.1)$$

$$\mathbb{R}^6 \xrightleftharpoons[\begin{smallmatrix} (\cdot)^\vee \end{smallmatrix}]{\begin{smallmatrix} (\cdot)^\wedge \end{smallmatrix}} \mathfrak{se}(3) \xrightleftharpoons[\log]{\exp} \text{SE}(3). \quad (5.2)$$

The hat and vee operators change representation without leaving the tangent space. The exponential and logarithmic maps move locally between a Lie algebra and its Lie group. The exponential map is many-to-one, so the logarithm requires a branch convention and is not globally unique.

5.1 Why is this useful in robotics?

Rotation matrices and rigid-body transformations are constrained. Adding an arbitrary matrix increment to $\mathbf{R} \in \text{SO}(3)$ will generally destroy orthogonality. A three-vector increment can instead be mapped through the exponential:

$$\mathbf{R}_{\text{new}} = \text{Exp}(\delta\phi)\mathbf{R}. \quad (5.3)$$

The result remains in $\text{SO}(3)$. Similarly,

$$\mathbf{T}_{\text{new}} = \text{Exp}(\delta\xi)\mathbf{T} \quad (5.4)$$

remains in $\text{SE}(3)$. This enables unconstrained local increments in nonlinear least squares, filtering, trajectory optimisation, visual odometry, and SLAM while preserving the geometry of the state.

6 Active and Passive Transformations

An *active* rotation changes a geometric vector while the coordinate frame remains fixed. If $\mathbf{Q}$ is the active rotation, then

$$\mathbf{x}' = \mathbf{Q}\mathbf{x}. \quad (6.1)$$

A *passive* transformation changes the coordinate frame used to describe the same geometric vector. If frame B is obtained by actively rotating frame A by $\mathbf{Q}$, then the coordinates satisfy

$${}^B\mathbf{x} = \mathbf{Q}^T {}^A\mathbf{x}. \quad (6.2)$$

Thus active and passive rotations use inverse matrices under this convention. The matrix ${}^w\mathbf{T}_b$ maps body-frame coordinates to world-frame coordinates,

$${}^w\tilde{\mathbf{p}} = {}^w\mathbf{T}_b {}^b\tilde{\mathbf{p}}, \quad (6.3)$$

but the notation alone does not make the transformation active or passive. That depends on whether the physical object or its coordinate description is being changed.

6.1 Left and right perturbations

Perturbation side is a separate choice from active versus passive interpretation. A left perturbation is

$$\mathbf{R}_{\text{new}} = \text{Exp}(\delta\phi_{\text{global}})\mathbf{R}, \quad (6.4)$$

where the increment is expressed in the destination or global frame. A right perturbation is

$$\mathbf{R}_{\text{new}} = \mathbf{R}\text{Exp}(\delta\phi_{\text{body}}), \quad (6.5)$$

where the increment is expressed in the source or body frame. The analogous SE(3) updates are

$$\mathbf{T}_{\text{new}} = \text{Exp}(\delta\xi_{\text{global}})\mathbf{T}, \quad \mathbf{T}_{\text{new}} = \mathbf{T}\text{Exp}(\delta\xi_{\text{body}}). \quad (6.6)$$

An algorithm must use one convention consistently when deriving Jacobians and covariances.

7 Implementation with Eigen

Eigen supplies fixed-size vectors and matrices but does not itself enforce Lie-group constraints. The following reusable functions construct the hat matrix, evaluate the $SO(3)$ exponential with a small-angle branch, and check a result. A standalone program would also need a test harness containing `main()`.

```
#include <Eigen/Core>
#include <Eigen/LU>
#include <cmath>

Eigen::Matrix3d hat(const Eigen::Vector3d& v) {
    Eigen::Matrix3d result;
    result << 0.0, -v.z(), v.y(),
              v.z(), 0.0, -v.x(),
              -v.y(), v.x(), 0.0;
    return result;
}

Eigen::Matrix3d expSO3(const Eigen::Vector3d& phi) {
    const double theta_squared = phi.squaredNorm();
    double a;
    double b;

    // Example double-precision threshold; tune for the application.
    if (theta_squared < 1e-12) {
        // Taylor series avoid cancellation and division by zero.
        a = 1.0 - theta_squared / 6.0
            + theta_squared * theta_squared / 120.0;
        b = 0.5 - theta_squared / 24.0
            + theta_squared * theta_squared / 720.0;
    } else {
        const double theta = std::sqrt(theta_squared);
        a = std::sin(theta) / theta;
        b = (1.0 - std::cos(theta)) / theta_squared;
    }

    const Eigen::Matrix3d phi_hat = hat(phi);
    return Eigen::Matrix3d::Identity()
        + a * phi_hat + b * phi_hat * phi_hat;
}

bool isRotation(const Eigen::Matrix3d& rotation) {
    const bool orthogonal =
        (rotation.transpose() * rotation)
        .isApprox(Eigen::Matrix3d::Identity(), 1e-12);
    return orthogonal
        && std::abs(rotation.determinant() - 1.0) < 1e-12;
}
```

The tolerance and small-angle threshold should be justified against the scalar type, expected input scale, and application accuracy. A production implementation should also test the identity, angles near π , round trips $\text{Log}(\text{Exp}(\phi))$, and invalid input matrices. Established manifold libraries can reduce implementation risk, but their ordering, perturbation, and frame conventions must still be checked.

Safety or certification suitability cannot be inferred from the mathematical correctness of a library. Such use requires the assurance evidence and third-party software process mandated by the specific project and standard.

8 Exercises

The following exercises progress from symbolic identities to frame reasoning and numerical validation. Use the notation and map conventions introduced in Section 1.1.

Exercise 8.1 (Hat and cross-product identities).

Let

$$\phi = \begin{bmatrix} \phi_x & \phi_y & \phi_z \end{bmatrix}^T.$$

Tasks

Show directly that $\phi^\wedge$ in (3.6) is skew-symmetric. Then show that $\phi^\wedge \mathbf{x} = \phi \times \mathbf{x}$ for arbitrary $\mathbf{x} \in \mathbb{R}^3$.

Exercise 8.2 (A quarter turn about the z-axis).

Consider

$$\phi = \begin{bmatrix} 0 & 0 & \pi/2 \end{bmatrix}^T.$$

Tasks

Compute $\phi^\wedge$ and use Rodrigues' formula to calculate $\mathbf{R} = \text{Exp}(\phi)$. Apply $\mathbf{R}$ to $\mathbf{e}_1$, $\mathbf{e}_2$, and $\mathbf{e}_3$, and describe the rotation geometrically.

Exercise 8.3 (Recovering a rotation vector).

Given

$$\mathbf{R} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

Tasks

Compute $\theta = \cos^{-1}((\text{tr}(\mathbf{R}) - 1)/2)$. Then calculate $(\mathbf{R} - \mathbf{R}^T)^\vee$ and use the logarithmic map to recover ϕ .

Exercise 8.4 (Inverse rigid-body transformation).

Consider

$${}^B\mathbf{T}_A = \begin{bmatrix} {}^B\mathbf{R}_A & {}^B\mathbf{t}_A \\ \mathbf{0}^T & 1 \end{bmatrix}.$$

Tasks

Derive its inverse and verify that ${}^B\mathbf{T}_A({}^B\mathbf{T}_A)^{-1} = \mathbf{I}_4$. Explain why the inverse translation is $-({}^B\mathbf{R}_A)^T {}^B\mathbf{t}_A$ rather than simply $-{}^B\mathbf{t}_A$.

Exercise 8.5 (Transforming a point between frames).

A point and translation are

$${}^A\mathbf{p} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^T, \quad {}^B\mathbf{t}_A = \begin{bmatrix} 2 & 3 & 0 \end{bmatrix}^T.$$

Frame A is rotated by 90° about the z -axis relative to frame B .

Tasks

Construct ${}^B\mathbf{T}_A$, calculate ${}^B\mathbf{p}$, and explain the result in terms of the two coordinate frames.

Exercise 8.6 (Composing a camera pose).

Let ${}^W\mathbf{T}_B$ map body-frame coordinates into the world frame, and let ${}^B\mathbf{T}_C$ map camera-frame coordinates into the body frame.

Tasks

Complete

$${}^w\mathbf{p} = (\dots)^C \mathbf{p}$$

and explain why the order of matrix multiplication is important.

Exercise 8.7 (Group and algebra classification).

Consider

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}.$$

Tasks

Identify representative quantities in $\text{SO}(3)$, $\mathfrak{so}(3)$, $\text{SE}(3)$, and $\mathfrak{se}(3)$. Explain the distinction between a Lie group and its Lie algebra.

Exercise 8.8 (The $\mathfrak{se}(3)$ hat operator).

Let

$$\boldsymbol{\xi} = \begin{bmatrix} \rho_x & \rho_y & \rho_z & \phi_x & \phi_y & \phi_z \end{bmatrix}^T.$$

Tasks

Construct $\boldsymbol{\xi}^\wedge$ and verify that

$$\boldsymbol{\xi}^\wedge = \begin{bmatrix} \phi^\wedge & \boldsymbol{\rho} \\ \mathbf{0}^T & 0 \end{bmatrix}.$$

Exercise 8.9 (Active and passive transformations).**Tasks**

A robot remains stationary while its coordinate frame is rotated by 45° . Classify the transformation and explain what happens to the physical point and its coordinates. Then classify the case in which the robot physically rotates by 45° while the world frame remains fixed.

Exercise 8.10 (Body-frame pose increment).

A current pose is ${}^w\mathbf{T}_B$, and an optimiser produces a small body-frame increment $\delta\boldsymbol{\xi}$.

Tasks

Determine whether $\text{Exp}(\delta\boldsymbol{\xi})$ should be applied on the left or right. Write the pose update and explain why multiplication order matters.

Exercise 8.11 (Manifold update versus matrix addition).

Let $\mathbf{R}_k \in \text{SO}(3)$ and $\delta\boldsymbol{\phi} \in \mathbb{R}^3$.

Tasks

Explain why

$$\mathbf{R}_{k+1} = \text{Exp}(\delta\boldsymbol{\phi})\mathbf{R}_k$$

preserves $\mathbf{R}_{k+1} \in \text{SO}(3)$, whereas

$$\mathbf{R}_{k+1} = \mathbf{R}_k + \delta\boldsymbol{\phi}^\wedge$$

does not generally produce a valid rotation matrix.

Exercise 8.12 (Exponential–logarithmic round trips).

Choose a rotation vector with $\|\boldsymbol{\phi}\| < \pi$ and a twist within the principal logarithm branch.

Tasks

Compute

$$\mathbf{R} = \text{Exp}(\boldsymbol{\phi}), \quad \boldsymbol{\phi}' = \text{Log}(\mathbf{R}),$$

and verify numerically that $\boldsymbol{\phi}' \approx \boldsymbol{\phi}$. Repeat for $\text{SE}(3)$ and verify $\text{Log}(\text{Exp}(\boldsymbol{\xi})) \approx \boldsymbol{\xi}$. Report the error norm and the tolerance used.

9 C++ Exercises

These implementation exercises build on one another. Retain the document's translation-first twist ordering and avoid Eigen's higher-level geometry types unless an exercise explicitly permits them.

Exercise 9.1 (Project and fixed-size types).

Create a C++ project using Eigen, a modern C++ standard, and strict compiler warnings.

Tasks

Represent $\mathbb{R}^3$ vectors, 3×3 matrices, 4×4 matrices, and six-vectors with fixed-size Eigen types. Do not use `AngleAxisd`, `Quaterniond`, or `Isometry3d`.

Exercise 9.2 (so(3) hat and vee operators).

```
Eigen::Matrix3d hat(const Eigen::Vector3d& phi);
Eigen::Vector3d vee(const Eigen::Matrix3d& Phi);
```

Tasks

Implement both functions and verify $((\phi)^\wedge)^\vee = \phi$ for several generated vectors. Also check that the hat result is skew-symmetric.

Exercise 9.3 (Rotation-matrix validation).

```
bool isRotationMatrix(const Eigen::Matrix3d& R,
                      double tolerance);
```

Tasks

Check $\mathbf{R}^T \mathbf{R} \approx \mathbf{I}_3$ and $\det(\mathbf{R}) \approx 1$. Test valid rotations, a reflection, and a matrix whose columns are not orthonormal.

Exercise 9.4 (SO(3) exponential map).

```
Eigen::Matrix3d expSO3(const Eigen::Vector3d& phi);
```

Tasks

Implement Rodrigues' formula from (3.10), including a stable small-angle branch. Verify the identity case and check that each result belongs to SO(3).

Exercise 9.5 (SO(3) logarithmic map).

```
Eigen::Vector3d logSO3(const Eigen::Matrix3d& R);
```

Tasks

Implement (3.13) and (3.14). Clamp the inverse cosine argument and handle rotations near zero and near π . Test principal branch round trips with justified tolerances.

Exercise 9.6 (SO3 class).

```
class SO3 {
public:
    SO3();
    explicit SO3(const Eigen::Matrix3d& R);
    Eigen::Matrix3d matrix() const;
    SO3 inverse() const;
    SO3 operator*(const SO3& other) const;
    Eigen::Vector3d operator*(
        const Eigen::Vector3d& point) const;
};
```

Tasks

Implement the operations from the mathematical definitions. Decide how the constructor handles an invalid matrix, and test identity, inverse, composition, and point rotation.

Exercise 9.7 (SE3 class).

```
class SE3 {
public:
    SE3();
    SE3(const Eigen::Matrix3d& R,
         const Eigen::Vector3d& t);
    Eigen::Matrix4d matrix() const;
    SE3 inverse() const;
    SE3 operator*(const SE3& other) const;
    Eigen::Vector3d operator*(
        const Eigen::Vector3d& point) const;
};
```

Tasks

Implement transformation construction, inversion, composition, and point transformation using (4.1) and (4.6).

Exercise 9.8 (se(3) hat and vee operators).

```
using Vector6d = Eigen::Matrix<double, 6, 1>;
Eigen::Matrix4d hat(const Vector6d& xi);
Vector6d vee(const Eigen::Matrix4d& Xi);
```

Tasks

Implement the translation-first convention in (4.7). Verify $((\xi)^\wedge)^\vee = \xi$ for representative inputs.

Exercise 9.9 (SE(3) exponential map).

```
SE3 expSE3(const Vector6d& xi);
```

Tasks

Implement (4.8) and the left Jacobian in (4.9). Include stable limits as $\theta \rightarrow 0$ and test pure translation, pure rotation, and combined motion.

Exercise 9.10 (SE(3) logarithmic map).

```
Vector6d logSE3(const SE3& T);
```

Tasks

Recover ϕ with the SO(3) logarithm and compute $\rho = \mathbf{J}(\phi)^{-1}\mathbf{t}$. Verify $\text{Log}(\text{Exp}(\xi)) \approx \xi$ within the principal branch.

Exercise 9.11 (Frame-chain round trip).

Create world, body, and camera frames with transformations ${}^W\mathbf{T}_B$ and ${}^B\mathbf{T}_C$.

Tasks

Transform several camera-frame points using

$${}^W\mathbf{p} = {}^W\mathbf{T}_B {}^B\mathbf{T}_C {}^C\mathbf{p}.$$

Transform them back into the camera frame with the inverses and report the round-trip error.

Exercise 9.12 (Numerical group properties).**Tasks**

Verify numerically

$$\begin{aligned} \mathbf{R}\mathbf{R}^{-1} &\approx \mathbf{I}_3, & \mathbf{T}\mathbf{T}^{-1} &\approx \mathbf{I}_4, \\ (\mathbf{R}_1\mathbf{R}_2)\mathbf{R}_3 &\approx \mathbf{R}_1(\mathbf{R}_2\mathbf{R}_3), & (\mathbf{T}_1\mathbf{T}_2)\mathbf{T}_3 &\approx \mathbf{T}_1(\mathbf{T}_2\mathbf{T}_3). \end{aligned}$$

Use justified numerical tolerances rather than exact equality.

Exercise 9.13 (Final reconstruction demonstration).

Generate a random pose whose rotation lies within the principal logarithm branch.

Tasks

Verify

$$\text{Exp}(\text{Log}(\mathbf{T})) \approx \mathbf{T}, \quad \text{Log}(\text{Exp}(\xi)) \approx \xi.$$

Report rotational and translational errors separately, along with the tolerances used to judge success.

10 Further Reading

References

- [1] T. D. Barfoot, *State Estimation for Robotics*, Cambridge University Press, 2017.
- [2] J. Solà, J. Deray, and D. Atchuthan, “A Micro Lie Theory for State Estimation in Robotics,” 2018. <https://arxiv.org/abs/1812.01537>
- [3] R. M. Murray, Z. Li, and S. S. Sastry, *A Mathematical Introduction to Robotic Manipulation*, CRC Press, 1994.
- [4] E. Eade, “Lie Groups for 2D and 3D Transformations,” 2017. <https://ethaneade.com/lie.pdf>